

# The Prompt Pack.

Everything you copy-paste into the Gemma assistant, the rules it won't remember for you, and the ritual that gets a score on the board. Built from your practice-round chats — then tested live against the real assistant on all eight task archetypes from past IOAI rounds. Every returned cell was run and verified. Six new traps found, all covered below.

# The assistant's real limits

Verified live on chat.ioai2026.kz. Every prompt in this pack is shaped around them.

| LIMIT        | VALUE              | WHAT IT MEANS FOR YOU                                         |
|--------------|--------------------|---------------------------------------------------------------|
| Reply length | ~2,000 tokens      | Ask for one short cell at a time — never "the whole solution" |
| Your message | 2,000 characters   | Paste long problem statements in parts (Prompt P1)            |
| Memory       | last 10 messages   | It forgets the problem — re-paste code when it gets confused  |
| Rate         | 60 messages / hour | ≈ 20 per problem. Never spend one on "thanks" or chit-chat    |

RULE #0 – ALWAYS PASTE, NEVER REFERENCE

Tested live: saying "check the same code again" without re-pasting it made the assistant invent a completely different problem (GPS distances!) and return garbage code — even though the code was still inside its 10-message window. Every prompt that talks about code must contain the code, pasted fresh. No "the code above", no "the same cell", no "it".

## Thirteen rules the model will not follow for you

**One chat per problem.** You pasted problem B's code into the clustering chat and lost two messages untangling it. Check the chat title before you paste.

**After any change: Run → Run All Cells.** Running only the edited cell caused your NameError: 'TRAIN' is not defined. Top to bottom, every time.

**Never modify the test data.** The assistant suggested rotating images inside the loader — that corrupts the test images at predict time and tanks the score. It was the planted trap in problem A. Augment nothing.

**The plan's #1 suggestion can be the trap.** Tested live: asked for improvements to problem A, the assistant's top suggestion was "data augmentation — rotations, flips, zooms." The exact planted bug. Check every suggestion against the Free-wins table; the table always wins.

**Always append the allowed-libraries line.** Left alone, it proposes CNNs, PyTorch and pip install — all banned or impossible (no internet in the notebook).

---

**Never say "give improved v2" with nothing attached.** Without your current code and score, it drifts into random changes. Paste both, ask for ONE change.

---

**If a code block looks cut off, use P6** — or better, re-ask for that one cell shorter. Never stitch fragments by hand.

---

**Two failed fixes → new chat.** Re-paste code + error fresh. Don't argue with a confused model.

---

**Never trust, always run.** Read the reply: does it use variables that exist? Does it invent files? Run it and check the self-score before believing it helped.

---

**Check your numbers survived.** Tested live: asked for a CNN on a 6-channel grid, got Conv2d(1, 32, ...) with a comment "assuming 1 channel" — crashes on the real data. Every number you specified must appear in the reply.

---

**Delete mock data before pasting.** Tested live: a fix came wrapped in "mock data structures to make it runnable" — a fake train set and a line that silently replaces your real labels with zeros. If the reply defines example/mock/placeholder data, delete those lines. P3 and P4 now forbid this, but stay alert.

---

**Old-API crashes are normal.** It wrote `df.append(...)` — removed from pandas years ago. Don't panic: P4 with the full traceback fixed it first try.

---

**Delete the stray "python" first line.** When copying code from the chat, the first line is sometimes just the word python — it's part of the formatting, not the code. Delete it or you get a SyntaxError.

## Seven steps, every problem, no exceptions

- 1 Read the problem once. Open its JupyterLab with the Server button on the contest page.

---

- 2 Open a new chat in the assistant. One chat = one problem.

---

- 3 Feed it the problem and baseline with P1, pasting in parts under 2,000 characters.

---

- 4 Submit the untouched baseline first. Run All Cells → Git tab → stage solution.ipynb → Commit → Push → contest page → Check. Points on the board before any improvement. Grading takes 5—9 minutes; confirm the score appears under Submissions.

---

- 5 Get the plan with P2 — then check every suggestion against the Free-wins table before acting. Apply exactly ONE improvement with P3.

---

- 6 Run All Cells, compare the self-score. Keep the change if better, revert if not. Errors → P4. Cut-off reply → P6. Two failed fixes → new chat.

---

- 7 Before the final push, run the safety check P7. Then commit → push → Check.

### Free wins — the eight archetypes, cram this table

The same task families keep coming back. These are the known winning moves — every row verified live on 2026-07-29/30: the assistant produced these cells from the pack's prompts, and each one was executed and scored. This table outranks anything P2 suggests, in both directions — P2 was excellent on robot cloning and useless on continual learning.

| IF THE TASK IS...                                                   | THE MOVE                                                                                                                                                                                       | PROOF                                              |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Clustering, features on different scales                            | StandardScaler before KMeans. Say the word "StandardScaler" in your P3 prompt.                                                                                                                 | ARI 0.56 → 1.00, one line                          |
| Small-image classification, sklearn only                            | Pixels scaled to 0—1, then SVC(kernel="rbf", C=10). No augmentation, no rotation.                                                                                                              | Accuracy 0.65 → ~0.84                              |
| Interactive / oracle task                                           | Run the shipped baseline as-is first. Heavy work belongs in __init__; ask questions that split the remaining candidates ~50/50; never eliminate a candidate on one surprising answer.          | 88% solved, ~12 queries avg                        |
| Continual learning — old classes + new classes, score averages both | Expand the head: new Linear(768, old+new), copy the old weight rows in. AND replay: pd.concat a slice of the ORIGINAL train.csv into the fine-tune data. Never fine-tune on new classes alone. | head-copy verified; forgetting is the planted trap |
| Behavioral cloning on grid observations (robot)                     | CNN on the grid — keep ALL its channels — concat the state vector, class weights 1/bincount for rare actions, invalid-action logits to -1e9 inside the training loss. Never BFS/A*/planning.   | plan matched official hints 3-for-3                |
| Match two sets of embeddings 1:1                                    | L2-normalize, cosine similarity matrix, scipy linear_sum_assignment(-sims). Never greedy argmax — it double-assigns.                                                                           | 100% on noisy pairs                                |
| Semi-supervised tabular (some rows labeled)                         | StandardScaler fit on labeled rows → LogisticRegression → predict unknown rows in original order.                                                                                              | 0.98 vs 0.90 unscaled                              |
| Char-level sequence labeling (word segmentation)                    | Context-window features, ONE dict key per offset ("c-3"... "c+3", "#" out of range) → DictVectorizer → LogisticRegression. One key per offset matters.                                         | F1 1.00 vs 0.66 (list-valued trap)                 |

And one meta-rule: the live leaderboard is hidden set A; your final score is hidden set B. Don't chase the live board — prefer simple, safe improvements.

# Copy, fill the {braces}, send

## P1 – FEED THE PROBLEM

Start of every problem. Paste statement + baseline in parts under 2,000 characters.

```
I am in a coding contest. I will paste the problem statement and the
baseline notebook code in several parts. Reply only "Noted." until I
write DONE.
```

...paste the parts, then:

```
DONE. Summarize in 5 short bullets: the input data, the exact output
file I must write, the scoring metric, the allowed libraries, and the
one thing most likely to go wrong.
```

## P2 – GET THE PLAN

After P1, before touching any code.

```
List the 3 changes to this baseline most likely to improve the score,
ordered by impact. Use ONLY these libraries: {allowed}.
No code yet, max 80 words.
```

### ⚠ TESTED LIVE – P2 CAN HAND YOU THE TRAP

For the digit problem, the assistant's #1 suggestion was "data augmentation (rotations, flips, zooms)" — the exact planted bug. Before acting on any P2 suggestion, check it against the Free-wins table. If P2 says augment / rotate / modify images or data: ignore it. The Free-wins table always outranks P2.

## P3 – REWRITE ONE CELL (THE WORKHORSE)

Your main tool. One cell, one change, per message. Always paste the cell.

```
Rewrite this exact notebook cell with ONE change: {the improvement}.
Rules: reply with ONLY the complete new cell in a single python code
block, max 50 lines, keep the same variable names and output format,
assume all variables and data already exist – do NOT create mock,
example, or placeholder data. Use ONLY these libraries: {allowed}.
My current cell:
{paste the cell}
```

#### P4 – FIX AN ERROR

Any red traceback. Paste the WHOLE error, not the last line.

My notebook cell crashed. Reply with ONLY the complete corrected cell in one python code block, no explanation. Assume all variables and data already exist – do NOT create mock, example, or placeholder data. Use ONLY these libraries: {allowed}.

The cell:  
{paste the cell}

The full error:  
{paste the whole red traceback}

#### P5 – IMPROVE THE SCORE

Baseline submitted, score in hand, want more.

My current self-score is {X}. Here is the code that produced it:  
{paste the key cells}

Suggest the ONE change most likely to raise the score and reply with only the complete modified cell. Do not change the output file name or format. Do not modify or augment the test data. Use ONLY: {allowed}.

#### P6 – REPLY GOT CUT OFF

Code block ends mid-line. Paste the last COMPLETE line.

Your code block was cut off. Continue EXACTLY from this line, code only, do not repeat anything before it:  
{paste the last complete line}

#### P7 – PRE-SUBMIT SAFETY CHECK

Always, right before your final push. The strict per-line format matters — tested live, the old wording returned a useless 'FAIL FAIL FAIL FAIL' on one line.

Check this notebook code against 4 points. For EACH point reply on its own line in exactly this format:  
"<point number>) PASS or FAIL – <quote the one code line that decides it>".

- 1) It never modifies/rotates/adds noise to test data before predicting.
- 2) It reads dataset/test\_public/ dynamically and never assumes answers.json exists there.
- 3) It writes {output file} in the working directory, one entry per test row.
- 4) It uses no library outside {allowed} and never needs internet.

Code:  
{paste all cells}

## P8 - EXPLAIN (WHEN LOST)

Cheap way to keep up with what you're running.

Explain what this cell does in max 5 simple sentences for a beginner:  
{paste}

### The submit ritual

Run All Cells → output file exists → Git tab → stage solution.ipynb → Commit → Push → contest page → Check → wait 5—9 minutes → score visible under Submissions. If your last submission is broken, fix it and resubmit — assume the latest one counts.